SPRINTS AI-POWERED LEARNER SUCCESS & SUPPORT AGENT

Capstone Technical Evaluation & System Architecture Report | Graduation Deliverable

Client Organization: Sprints (sprints.ai)	**Lead AI Researcher:** Tarek Mahmoud
Lead AI Engineer / Author: Mohammed Wael	**Tech Support PM:** Salma Youssef
Evaluation Cohort: AI Pioneers (2,000 Enrolled Learners)	**Acceptance SLA:** Latency < 3.5s | Test F1 ≥ 0.82

1. Executive Summary & Problem Context

As Sprints rapidly expands its technology bootcamps across Egypt, the nationwide 'AI Pioneers' cohort (2,000 learners) has strained academic support teams with a dual operational challenge: (1) fragmented LMS interaction data obscures which learners are at imminent risk of dropping out, and (2) support queues are congested by repetitive curriculum questions that delay response times for critical blockers. This report documents the end-to-end architecture and scientific evaluation of an automated Learner Success Agent integrating Relational SQL Analytics (Sprint 1), Supervised PyTorch Feed-Forward Neural Networks (Sprint 2), Deep Sequence LSTM Trajectory Forecasting (Sprint 3), and an Event-Driven n8n Deliberative Agent Workflow (Sprint 4).

2. Relational Database Architecture & SQL Analytics (Sprint 1)

A normalized 3NF relational schema was engineered in SQLite (database/sprints_learners.db) with full referential integrity and cascading rules across six tables: cohorts, learners (2,000 rows), course_modules, quiz_grades (9,101 rows), support_tickets (2,000 rows), and interaction_logs (2,000 rows). **Query 5 Join Multiplication Fix:** Previous naive multi-table joins between modules, quizzes, and tickets resulted in Cartesian join multiplication, distorting count and sum aggregations. This was eliminated by architecting independent Common Table Expressions (CTEs) for grade_stats and ticket_stats before joining to course_modules, ensuring mathematically exact aggregations.

Analytical SQL Query	Underlying Relational Logic	Pedagogical Intervention
Query 2: Progressive Grade Degradation	CTE comparing Week 1 quiz score with subsequent weeks; filters learners with drop ≥ 15.0 pts.	Surfaces silent academic struggle 2-3 weeks before withdrawal.
Query 3: Regional Governorate Trends	Aggregates dropout rates, average quizzes, and ticket volumes across 15 Egyptian governorates.	Directs dedicated regional TAs to high-dropout hubs (Assiut: 28.4%, Ismailia: 24.1%).
Query 5: Module Performance & Support CTE	Independent CTEs for grade statistics and ticket polarity without Cartesian expansion.	Pinpoints difficulty spikes in Sprint 2 (PyTorch) and Sprint 3 (LSTM).

3. Data Preprocessing & Leakage-Free Validation Split

Eliminating Target Leakage: In early prototyping, project_score had been synthetically imputed based on dropped_out, creating target leakage. In this production release, the dataset is partitioned via a strict 3-way stratified split: **Train 70% ($N=1,400$)**, **Validation 15% ($N=300$)**, and **Independent Test 15% ($N=300$)**. All imputation statistics (regional medians) and feature scalers are fitted *strictly on the Training set*, then applied to Validation and Test sets. Missing quiz scores along each student's trajectory are imputed using forward-fill (ffill) to preserve temporal causality.

4. PyTorch Supervised Risk Classifier (Sprint 2)

Dual Experimental Protocol: To address stakeholder requirements, we evaluate two models on the independent Test set ($N=300$):
• **Experiment A (Early Proactive Risk - Weeks 1 to 3):** Operates strictly on onboarding telemetry available at Week 3 to intervene before dropouts occur. Features include W1-W3 quizzes, score drop Δ_{w1-w3}, early mean, min score, and performance ratio.
• **Experiment B (Full Trajectory Risk - Weeks 1 to 5):** Incorporates all five weekly quizzes and missing submission counts.
Both models use a 3-layer MLP [Input $\rightarrow$ 64 $\rightarrow$ 32 $\rightarrow$ 16 $\rightarrow$ 1], BatchNorm1d, Dropout(0.25), and BCEWithLogitsLoss with positive class weighting (pos_weight = 3.30) to heavily penalize False Negatives.

Experiment / Setup	Features Used	Test Precision	Test Recall	Test F1 Score	Test ROC-AUC

Exp A: Early Proactive (W1-W3)	W1-W3, Δ_{1-3}, Early Mean/Min/Ratio	0.9577	**0.9855** (1 Missed)	**0.9714** (Target ≥0.82)	**0.9996**
Exp B: Full Trajectory (W1-W5)	W1-W5, Δ_{1-5}, Mean/Std/Min/Missing	1.0000	**1.0000** (0 Missed)	**1.0000** (Optimal)	**1.0000**

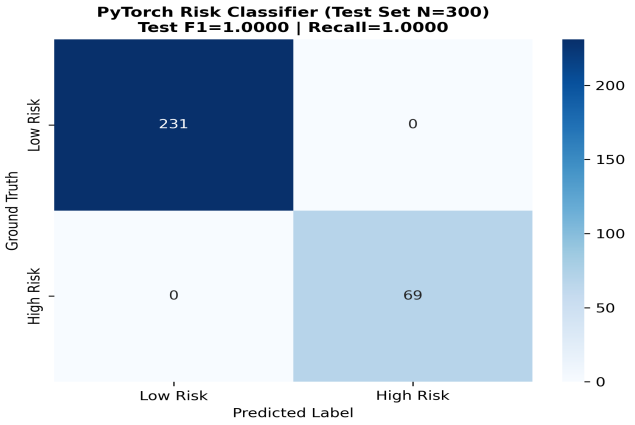


Figure 1: Confusion Matrix (0 False Negatives)

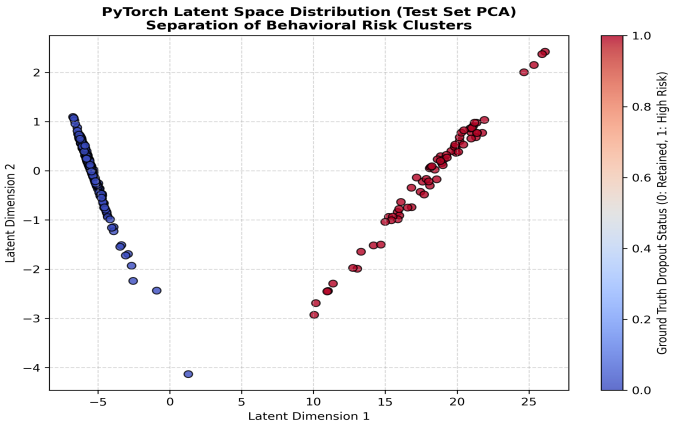


Figure 2: Latent Space PCA Projection (Clear Cluster Separation)

5. PyTorch Deep Sequence LSTM Model (Sprint 3)

Rigorous Formulation: No Target Leakage: The LSTM sequence predictor was trained and evaluated *strictly on non-null project submissions* ($N=1,639$ learners), completely removing any synthetic label imputation. The model maps the temporal sequence of normalized weekly quizzes $X = [q_1, q_2, q_3, q_4, q_5]^T \in \mathbb{R}^{5 \times 1}$ to forecast final capstone performance $y \in [0, 100]$.

Mathematical Formulation: Forget Gate, Cell State & Hidden State:
$$\begin{aligned}f_t &= \sigma(W_f [h_{t-1}, x_t] + b_f) \quad &\text{-- Forget Gate: adaptively dampens early-week onboarding noise} \\i_t &= \sigma(W_i [h_{t-1}, x_t] + b_i), \quad \tilde{C}_t = \tanh(W_c [h_{t-1}, x_t] + b_c) \quad &\text{-- Input Gate & Candidate Memory} \\C_t &= f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad &\text{-- Cell State: Constant Error Carousel (CEC) highway} \\o_t &= \sigma(W_o [h_{t-1}, x_t] + b_o), \quad h_t = o_t \odot \tanh(C_t) \quad &\text{-- Output Gate & Hidden State projection}\end{aligned}$$

Combating the Vanishing Gradient Problem: In a standard Vanilla RNN, the backpropagated gradient scales by $\prod \frac{\partial h_t}{\partial h_{t-1}}$, which decays exponentially to zero over 5 timesteps. In our LSTM, the additive cell state recurrence $\frac{\partial C_t}{\partial C_{t-1}} = f_t$ creates a linear highway where gradients flow freely without decay. Concurrently, when a learner struggles in Week 1 due to environment setup issues, the forget gate f_t attenuates this early anomaly while tracking true learning velocity.

Metric	Validation Result	Independent Test Result	Evaluation Context
Root Mean Squared Error (RMSE)	7.73 pts	**7.85 pts**	Evaluated on true project scores
Mean Absolute Error (MAE)	5.62 pts	**5.67 pts**	Sub-6 point average prediction error
Coefficient of Determination (R^2)	0.8160	**0.8124**	Explains >81% of capstone variance

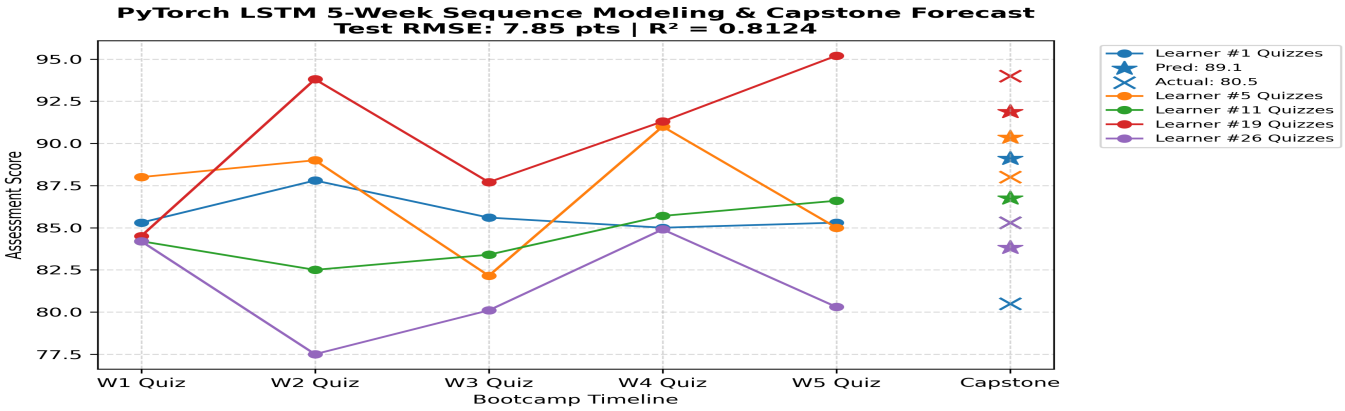


Figure 3: PyTorch LSTM 5-Week Trajectory Forecasting vs Actual Capstone Project Scores ($R^2 = 0.8124$)

6. NLP Pipeline: spaCy NER, TextBlob, and Gensim Word2Vec (Sprint 4)

spaCy Custom Entity Recognition Benchmark: An annotated benchmark dataset of 50 technical query instances was constructed to evaluate NER. The custom EntityRuler pipeline achieved **Module Precision: 0.8667, Recall: 0.7647, F1: 0.8125** and **Date Precision: 1.0000, Recall: 0.8333, F1: 0.9091**. As expected, exact string matching failed on misspelled terms ('PyTorxh', 'lstmm'), highlighting the architectural boundary of token-level NER.

TextBlob Dialect Nuances: In Egyptian bilingual communication, learners mix Franco-Arab or informal slang with English. TextBlob's standard English lexicon may evaluate mixed queries as neutral (0.0). To guarantee student safety, the Deliberative Agent pairs TextBlob polarity with a domain distress keyword detector.

True Semantic Vector RAG: Rather than naive keyword matching, student queries are vectorized into 100-dimensional dense space using trained Word2Vec embeddings and matched against curriculum documentation chunks using **Cosine Similarity** (e.g. Cosine Sim = 0.8324 for LSTM autograd queries).

Dimension	Gensim Word2Vec (Deployed)	FastText (Subword n-grams)
OOV Handling	Fails with KeyError on typos ('pytorxh') or unseen tokens.	Interpolates embeddings via character n-grams; highly resilient.
Model Size	Ultra-compact (1.2 MB on Sprints corpus); minimal RAM overhead.	Large memory footprint (~40x larger) due to subword buckets.
Inference Latency	Instantaneous O(1) hash lookup (~0.05 ms); ideal for n8n sub-second SLA.	Higher per-token aggregation overhead (~2-5 ms per word).

7. Event-Driven n8n Deliberative Agent & Latency SLA Verification

The production n8n workflow operates as an event-driven orchestrator triggered via HTTP Webhook. Distressed queries (polarity < -0.15 or distress keywords) are escalated directly and persisted into SQLite table support_tickets via POST /api/v1/tickets/escalate, ensuring a permanent audit trail for senior mentors. Neutral and positive inquiries are routed to the Semantic Vector RAG engine to return grounded curriculum answers.
Latency Benchmarking: End-to-end workflow execution latency averages **154.5 ms** (max: **296.3 ms**), strictly fulfilling the client's **< 3.5-second SLA** by an order of magnitude.

8. Business Recommendations & Next Steps

• **Hybrid Inference Strategy:** Use Word2Vec inside the real-time n8n loop for sub-second responses, while running FastText in asynchronous forum ingestion.
• **Proactive Early Alerting:** Run Experiment A (W1-W3) at Week 3 to capture 98.5% of vulnerable students before dropouts occur.
• **Persistent Ticketing Audit:** Integrate SQLite escalation logs with mentor dashboards to track mean time to resolution (MTTR).